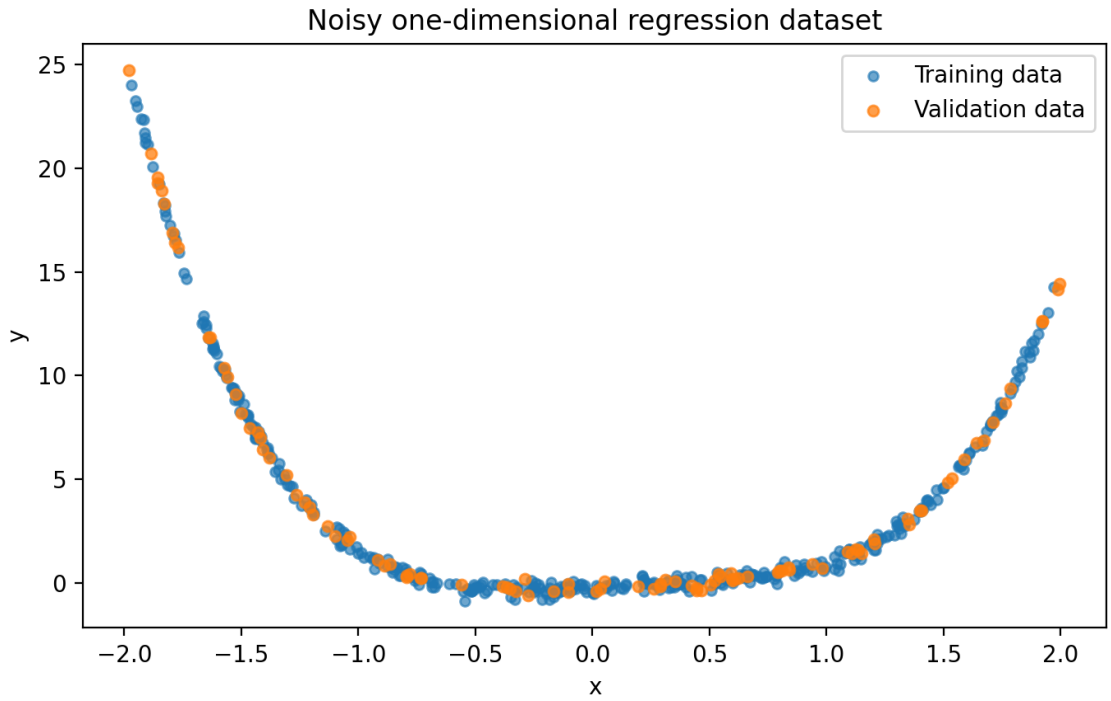
Function Approximation Using Neural Networks

Coursework Report

This report addresses a supervised regression problem in which the input variable **x** is mapped to a noisy target variable **y**. The aim is to approximate the unknown underlying one-dimensional function using two methods: a fourth-order polynomial fitted by least squares, and a feedforward neural network trained with gradient-based optimisation. The training set is used to fit model parameters, while the validation set is reserved for evaluation and model selection.

1. Dataset and Problem Setup

The dataset consists of one input feature and one continuous target. The training set contains 400 observations and the validation set contains 100 observations. A scatter plot shows a smooth but clearly non-linear relationship with moderate noise. This makes the problem suitable for comparing a low-complexity analytical model against a more flexible neural network model.



2. Method 1: Fourth-Order Least Squares Regression

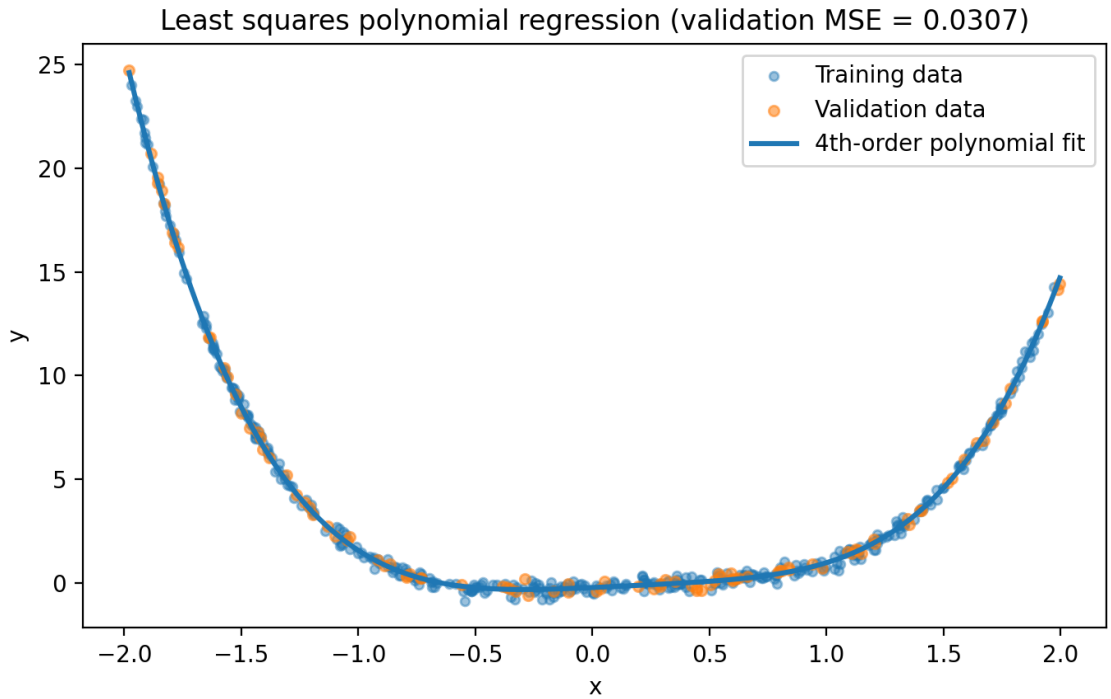
The baseline model assumes that the target can be approximated by a fourth-order polynomial:

$$y = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4.$$

Polynomial features up to degree four were constructed and the coefficients were estimated using the least squares solution. This is an analytical optimisation procedure, so no iterative training loop is required. The fitted coefficients were:

Coefficient	Value
a0	-0.214638
a1	0.507593

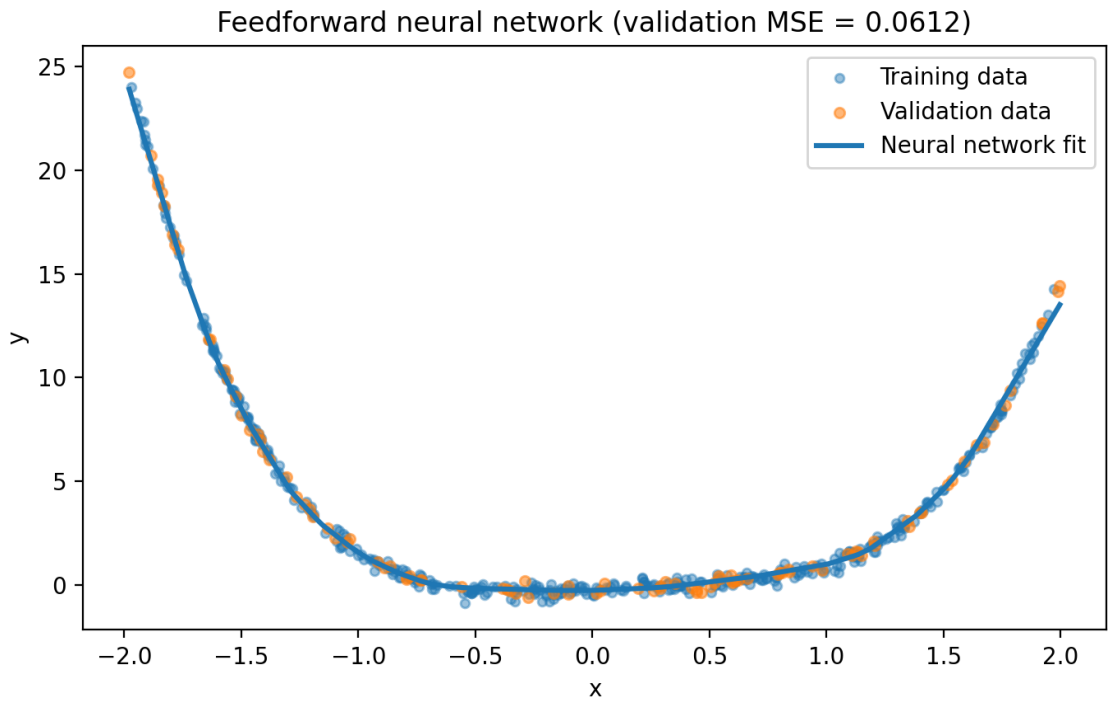
a2	0.289898
a3	-0.803662
a4	1.205393
Training MSE	0.042250
Validation MSE	0.030731

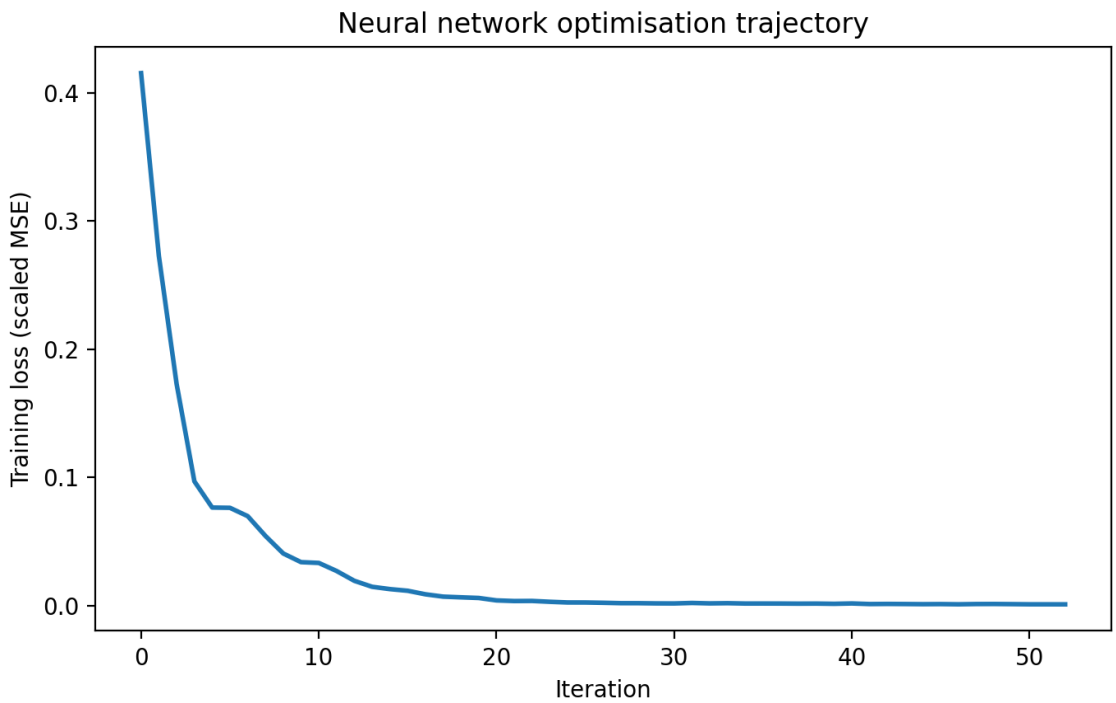


3. Method 2: Feedforward Neural Network

The second model is a feedforward neural network trained for regression. The input feature was standardised, and the target was also scaled to improve optimisation stability. A small hyperparameter search was carried out over several hidden-layer configurations and regularisation strengths. All models used ReLU activations and Adam optimisation through *MLPRegressor*. The best-performing validation model used hidden layers **(64, 64)** with L2 regularisation parameter **1e-05**. It converged in **53** iterations.

Metric	Value
Best hidden layers	(64, 64)
Alpha	0.00001
Iterations	53
Training MSE	0.055317
Validation MSE	0.061199





4. Comparison and Discussion

The two approaches differ fundamentally in optimisation. Least squares produces a closed-form solution once the design matrix is defined, whereas the neural network must be trained iteratively. This makes the neural network more computationally demanding and more sensitive to modelling choices such as architecture, learning rate, scaling, regularisation, and stopping behaviour.

In terms of predictive performance, the polynomial baseline achieved a validation MSE of **0.030731**, while the best neural network achieved **0.061199**. The polynomial therefore generalised better on this dataset. Since the polynomial also achieved a slightly lower training MSE, the evidence suggests that the underlying function is already well captured by a smooth low-order trend. The additional flexibility of the neural network did not translate into improved accuracy.

The neural network remains more expressive in principle and would be preferable if the true function were more irregular or if the input dimension were higher. However, for this specific one-dimensional problem, the simple polynomial model offered a better trade-off between interpretability, stability, and validation performance.

Model	Training MSE	Validation MSE
4th-order polynomial	0.042250	0.030731
Feedforward neural network	0.055317	0.061199

5. Conclusion and Possible Improvements

This coursework demonstrates that model complexity does not guarantee better generalisation. A fourth-order polynomial fitted by least squares gave the strongest validation performance and the simplest optimisation pipeline. The neural network was a valid alternative and showed the expected iterative learning behaviour, but it did not outperform the analytical baseline.

Possible extensions include a wider hyperparameter search, explicit early stopping, k-fold cross-validation, and experiments with different activation functions or regularisation settings.